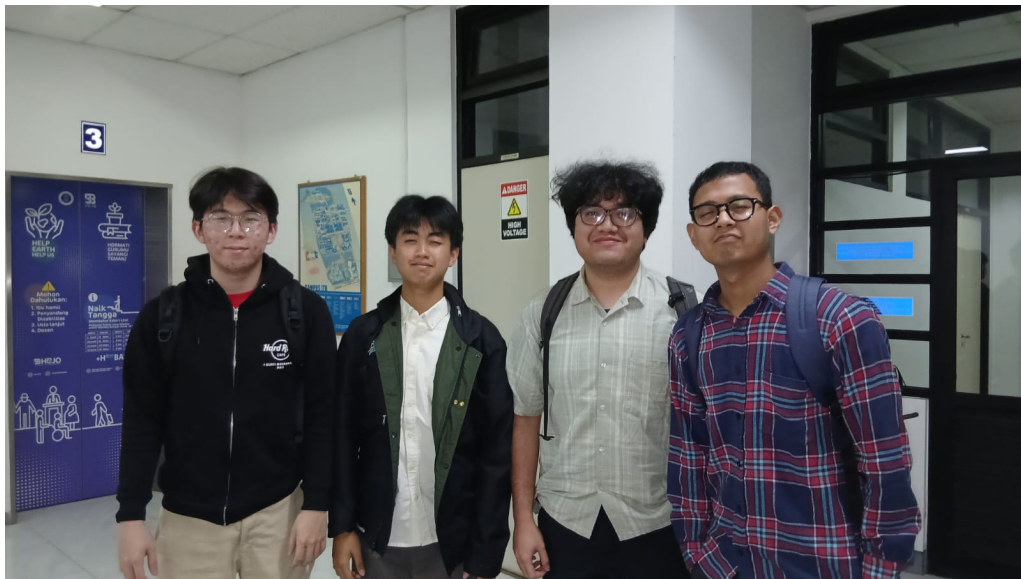


Tugas Besar

Milestone 4

IF2224 Teori Bahasa Formal dan Otomata
Intermediate Code and Interpreter
Semester 2 Tahun 2025/2026



Disusun oleh:

Kai

Jonathan Kris Wicaksono	(13524023)
Vincent Rionarlie	(13524031)
Muhammad Aufar Rizqi Kusuma	(13524061)
Bryan Pratama Putra Hendra	(13524067)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	4
1.1	Latar Belakang	4
1.2	Rumusan Masalah	4
1.3	Tujuan	5
1.4	Batasan Masalah	5
2	Dasar Teori	6
2.1	Intermediate Code Generation	6
2.2	Three-Address Code	7
2.3	Instruction Set	7
2.4	Arsitektur Mesin Eksekusi: Stack Machine	9
2.4.1	Siklus Fetch-Decode-Execute	9
2.4.2	Memori Eksekusi: The Stack	10
2.5	Translasi Control Flow	10
2.5.1	Translasi IF-ELSE	10
2.5.2	Translasi WHILE Loop	11
2.5.3	Translasi FOR Loop	11
2.5.4	Translasi CASE	12
2.6	Runtime Vulnerabilities	12
2.6.1	Kerentanan pada Stack	12
2.6.2	Kerentanan Memori dan Akses	12
2.6.3	Kerentanan Alur Eksekusi	12
2.6.4	Kerentanan Aritmatika	12
3	Perancangan dan Implementasi	14
3.1	Gambaran Umum Program	14
3.2	Struktur Direktori Program	15
3.3	Implementasi Intermediate Code Generator	15
3.3.1	Inisialisasi Memori	16
3.3.2	Translasi Statement	16
3.3.3	Translasi Ekspresi	16
3.3.4	Translasi Procedure Call	16
3.3.5	Backpatching	17
3.4	Implementasi Interpreter	17
3.4.1	Siklus Eksekusi	17
3.4.2	Manajemen Stack	17



3.4.3	Frame Management	18
3.4.4	Validasi Keamanan	18
3.5	Format Output	18
4	Pengujian	19
4.1	Strategi Pengujian	19
4.2	Pengujian Lintas Milestone	20
4.3	Detail Pengujian	20
4.3.1	Kasus 1: Operasi Dasar	20
4.3.2	Kasus 2: Control Flow Bersarang	21
4.3.3	Kasus 3: Perulangan While	23
4.3.4	Kasus 4: Perulangan For	24
4.3.5	Kasus 5: Case Statement	26
4.3.6	Kasus 6: Array Dua Dimensi	27
4.3.7	Kasus 7: Runtime Error Division by Zero	28
4.4	Pengujian Error Runtime	29
5	Penutup	30
5.1	Kesimpulan	30
5.2	Saran	30
6	Lampiran	32
6.1	Tautan GitHub	32
6.2	Tabel Kontribusi	33
6.3	Cara Menjalankan Program	33



Daftar Algoritma

1	Siklus Fetch-Decode-Execute	17
---	---------------------------------------	----

Bab 1

Pendahuluan

1.1 Latar Belakang

Setelah melewati tahap lexical analysis, syntax analysis, dan semantic analysis, compiler telah memiliki representasi program yang valid secara sintaksis maupun semantik dalam bentuk decorated AST dan symbol table. Namun, decorated AST masih berbentuk pohon yang tidak dapat dieksekusi secara langsung oleh mesin. CPU dan memori komputer dirancang untuk mengeksekusi instruksi secara sekuensial, bukan dengan melompat-lompat dalam struktur pohon yang kompleks. Oleh karena itu, diperlukan sebuah jembatan yang meratakan decorated AST menjadi daftar instruksi sederhana dan sekuensial yang disebut intermediate code.

Pada Milestone 1, Arion Compiler telah memiliki lexer yang mengubah source code menjadi token stream. Pada Milestone 2, token tersebut diproses oleh Recursive Descent Parser untuk menghasilkan parse tree. Pada Milestone 3, parse tree dikonversi menjadi AST, dianalisis secara semantik, dan diperkaya dengan anotasi tipe data, scope, serta referensi ke symbol table. Hasil akhirnya adalah decorated AST beserta tiga tabel simbol `tab`, `btab`, dan `atab`.

Milestone 4 berfokus pada dua komponen terakhir dalam pipeline compiler Arion, yaitu intermediate code generation dan interpreter. Intermediate code generator bertugas mengubah decorated AST menjadi daftar instruksi standar yang mirip dengan bahasa mesin namun masih mudah dibaca manusia. Instruksi-instruksi ini kemudian dieksekusi oleh interpreter yang berperan sebagai virtual machine. Interpreter membaca instruksi satu per satu, mengelola memori melalui stack, mengevaluasi operasi matematika, mengatur alur kontrol program, dan menghasilkan output nyata ke layar.

1.2 Rumusan Masalah

Permasalahan utama pada Milestone 4 adalah bagaimana membangun intermediate code generator dan interpreter yang dapat memanfaatkan decorated AST hasil Milestone 3 untuk menghasilkan output final dari sebuah program Arion. Permasalahan tersebut diuraikan menjadi beberapa pertanyaan sebagai berikut. Pertama, bagaimana mengonversi decorated AST menjadi intermediate code berupa daftar

instruksi sekuensial menggunakan pendekatan Three-Address Code. Kedua, bagaimana merancang instruction set yang mencakup operasi load, store, aritmatika, perbandingan, jump, call, dan return. Ketiga, bagaimana mengimplementasikan stack machine untuk mengelola memori eksekusi, termasuk stack frame, static link, dynamic link, dan return address. Keempat, bagaimana melakukan translasi control flow (`if`, `while`, `for`, `repeat`, `case`) ke dalam instruksi `JMP` dan `JPC` dengan label. Kelima, bagaimana menangani kerentanan runtime seperti stack overflow, out-of-bounds access, invalid jump target, dan numerical overflow. Keenam, bagaimana menampilkan output intermediate code dan hasil eksekusi program secara terformat.

1.3 Tujuan

Tujuan pengerjaan Milestone 4 adalah membangun tahap intermediate code generation dan interpreter pada Arion Compiler. Secara khusus, tujuan yang ingin dicapai adalah membuat intermediate code generator yang dapat mengubah decorated AST menjadi daftar instruksi `LIT`, `LOD`, `STO`, `INT`, `JMP`, `JPC`, `OPR`, `CAL`, dan `RET`. Selain itu, diimplementasikan pula stack machine dengan siklus fetch-decode-execute untuk menjalankan intermediate code, pengelolaan memori eksekusi melalui stack termasuk alokasi frame, push-pop operand, dan pengaturan stack frame untuk procedure call, penanganan translasi control flow dari struktur pohon menjadi instruksi sekuensial dengan label dan jump, penambahan deteksi dan penanganan runtime vulnerabilities pada interpreter, serta penghasilan output berupa intermediate code dan output eksekusi program sebagai hasil akhir compiler.

1.4 Batasan Masalah

Implementasi Milestone 4 dibatasi pada backend compiler Arion. Program tidak lagi melakukan analisis sintaksis atau semantik, karena validitas program diasumsikan sudah diperiksa oleh tahap sebelumnya. Intermediate code generator bekerja pada decorated AST yang dibangun dari parse tree hasil parser, sedangkan interpreter mengeksekusi intermediate code tanpa mempedulikan sintaks bahasa sumber.

Pengecekan runtime yang diimplementasikan berfokus pada stack overflow, stack underflow, stack smashing, out-of-bounds array access, invalid jump target, dan numerical overflow atau underflow. Fitur yang belum didukung pada tahap ini meliputi function call dalam ekspresi (`FuncCall`), record field access dalam intermediate code, serta boolean operator `and`, `or`, dan `not` pada level code generation.

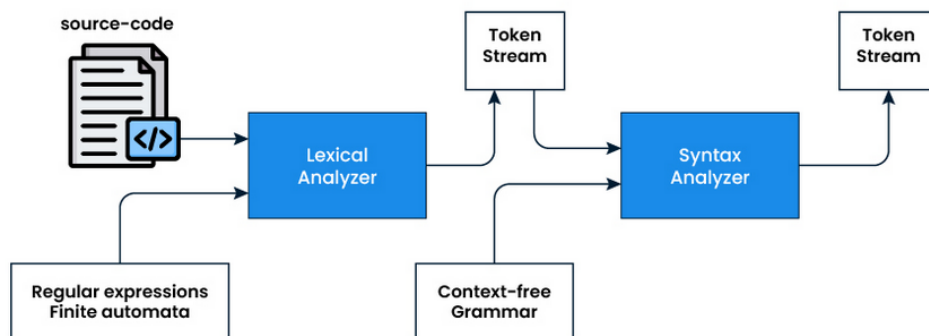
Bab 2

Dasar Teori

2.1 Intermediate Code Generation

Intermediate code generation adalah tahap backend compiler yang mengubah representasi pohon (decorated AST) menjadi daftar instruksi sekuensial yang dapat dieksekusi oleh mesin virtual. Tahap ini tidak lagi mempedulikan sintaks bahasa sumber, melainkan fokus pada sintesis dan komputasi. Tujuannya adalah meratakan struktur pohon yang rumit menjadi instruksi-instruksi primitif yang sangat sederhana [1, 3].

Dalam arsitektur compiler modern, pemisahan frontend dan backend sangat penting. Frontend berurusan dengan source code, token, parse tree, dan semantic analysis. Backend berfokus pada konversi decorated AST menjadi intermediate code dan eksekusi oleh interpreter. Dengan desain ini, jika suatu saat bahasa Arion memiliki sintaks baru, hanya frontend yang perlu dirombak selama decorated AST yang dihasilkan tetap sama [2].



Gambar 2.1: Alur umum intermediate code generation dalam compiler

Gambar 2.1 menunjukkan bahwa intermediate code generation berada setelah semantic analysis dan sebelum eksekusi oleh interpreter. Pada posisi ini, compiler mulai mengonversi pemahaman konseptual program menjadi instruksi operasional.

2.2 Three-Address Code

Three-Address Code (TAC) adalah format intermediate code yang paling luas digunakan di industri. Dinamakan “three-address” karena setiap baris instruksi didesain untuk sangat sederhana, di mana maksimal hanya boleh melibatkan tiga alamat memori: dua operand sumber dan satu lokasi tujuan [1]. Struktur dasar TAC terlihat seperti berikut.

Struktur Dasar TAC

```
hasil = operand1 operator operand2
```

Dalam bahasa tingkat tinggi, programmer dapat menulis ekspresi matematis yang sangat panjang dan kompleks. Namun, CPU tidak bisa menghitung semuanya sekaligus. CPU harus memecahnya menjadi operasi-operasi biner yang sederhana. TAC menyimulasikan batasan CPU ini dengan menggunakan variabel sementara, biasanya direpresentasikan sebagai `t1`, `t2`, `t3`, dan seterusnya.

Sebagai contoh, ekspresi `x := (a + b) * (c - d)` akan dipecah menjadi empat instruksi TAC.

Contoh TAC

```
t1 := a + b
t2 := c - d
t3 := t1 * t2
x  := t3
```

Dengan memecah ekspresi menggunakan TAC, kompleksitas kode tingkat tinggi telah berhasil diturunkan menjadi barisan instruksi primitif yang sangat mudah dieksekusi oleh mesin.

2.3 Instruction Set

Instruction set yang digunakan pada Arion Compiler terdiri dari sembilan instruksi utama. Setiap instruksi memiliki opcode, level, dan operand. Level merepresentasikan lexical level untuk akses variabel non-lokal, sedangkan operand menyimpan nilai literal, alamat memori, atau nomor operasi.

Tabel 2.1: Daftar instruksi intermediate code

Instruksi	Operasi	Keterangan
LIT <i>v</i>	Load Literal	Memasukkan nilai literal <i>v</i> ke dalam stack.
LOD <i>a</i>	Load Value	Memuat nilai dari alamat <i>a</i> .
STO <i>a</i>	Store Value	Menyimpan nilai ke alamat <i>a</i> .
CAL <i>l</i>	Call	Memanggil fungsi di garis <i>l</i> .
INT <i>m</i>	Initiate Memory	Membuat memori dengan ukuran <i>m</i> .
JMP <i>l</i>	Unconditional Jump	Lompat ke garis <i>l</i> tanpa kondisi.
JPC <i>l</i>	Conditional Jump	Lompat ke garis <i>l</i> bila kondisi tidak memenuhi.
OPR <i>o</i>	Operation	Memanggil operasi <i>o</i> (aritmatika, logika, I/O).
RET	Return	Keluar dari fungsi atau prosedur.

Instruksi OPR memiliki sub-operasi yang lebih spesifik. Sub-operasi yang didukung meliputi negasi, penjumlahan, pengurangan, perkalian, pembagian, modulo, perbandingan (equal, not equal, less, greater, less-equal, greater-equal), serta operasi I/O WRT dan WRTLN.

Tabel 2.2: Daftar sub-operasi instruksi OPR

No	Operation Name	Function
1	NEG	Negasi nilai teratas dari stack.
2	ADD	Menambah dua nilai teratas dari stack.
3	SUB	Mengurangi dua nilai teratas dari stack.
4	MUL	Mengali dua nilai teratas dari stack.
5	DIV	Membagi dua nilai teratas dari stack.
6	MOD	Modulus dua nilai teratas dari stack.
7	EQL	Memeriksa kesamaan dua nilai teratas.
8	NEQ	Memeriksa ketidaksamaan dua nilai teratas.
9	LSS	Memeriksa apakah nilai pertama kurang dari kedua.
10	GEQ	Memeriksa apakah nilai pertama lebih dari sama dengan kedua.
11	GTR	Memeriksa apakah nilai pertama lebih dari kedua.
12	LEQ	Memeriksa apakah nilai pertama kurang dari sama dengan kedua.
13	WRT	Menulis output tanpa newline.
14	WRTLN	Menulis output dengan newline.

2.4 Arsitektur Mesin Eksekusi: Stack Machine

Interpreter pada Arion Compiler bertindak sebagai virtual machine (VM) yang mengeksekusi intermediate code. VM ini tidak memiliki bentuk fisik, tetapi memiliki logika memori dan siklus kerja yang sama persis dengan CPU sungguhan.

2.4.1 Siklus Fetch-Decode-Execute

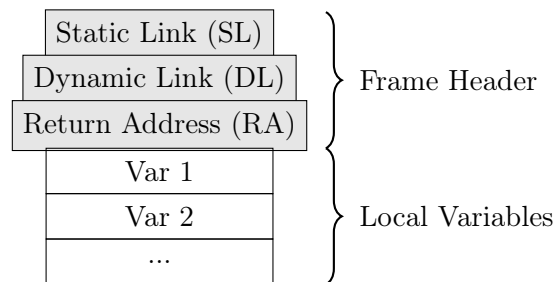
Sama seperti CPU sungguhan, interpreter membaca instruksi satu per satu menggunakan penunjuk khusus yang disebut Instruction Pointer (IP) atau Program Counter (PC). Interpreter beroperasi dalam siklus eksekusi yang terdiri dari tiga tahap utama [3]. Tahap pertama adalah **Fetch**, yaitu membaca baris instruksi TAC yang saat ini ditunjuk oleh IP. Tahap kedua adalah **Decode**, yaitu membedah baris tersebut untuk mengetahui perintahnya dan operand-nya. Tahap ketiga adalah **Execute**, yaitu melakukan aksi nyata, seperti menghitung nilai operasi matematika atau menyimpan nilai ke memori.

Setelah tahap execute selesai, IP akan bergerak maju ke baris instruksi berikutnya, kecuali jika instruksi tersebut adalah **JMP** atau **JPC**, di mana IP akan melompat ke nomor baris yang dituju.

2.4.2 Memori Eksekusi: The Stack

Untuk bisa melakukan perhitungan dan menyimpan variabel, VM membutuhkan memori. Struktur memori yang paling vital adalah **stack**. Stack bekerja dengan prinsip LIFO (Last In, First Out). Mengapa menggunakan stack dan bukan array biasa? Jawabannya berkaitan erat dengan scope dan function call.

Setiap kali program memasuki blok kode baru atau memanggil fungsi, interpreter akan membuat kotak memori baru di dalam stack yang disebut **stack frame**. Stack frame menyimpan semua variabel lokal milik fungsi tersebut, argumen yang di-passing, serta return address. Saat fungsi dipanggil, stack frame baru di-push ke pucuk tumpukan. Ketika fungsi selesai, frame tersebut di-pop dari stack, dan semua variabel lokalnya otomatis musnah.



Gambar 2.2: Ilustrasi stack frame dengan static link, dynamic link, dan return address

Setiap stack frame memiliki tiga slot header yang selalu diisi. **Static Link** pada slot 0 menunjuk ke base pointer frame induk secara statis berdasarkan lexical level. **Dynamic Link** pada slot 1 menunjuk ke base pointer frame pemanggil pada runtime. **Return Address** pada slot 2 menyimpan alamat instruksi yang harus dieksekusi setelah fungsi selesai. Slot-slot setelah header digunakan untuk variabel lokal dan operand sementara selama eksekusi.

2.5 Translasi Control Flow

Mengubah ekspresi matematika menjadi TAC relatif mudah karena polanya pasti. Tantangan terbesar adalah meratakan control flow seperti **if-else** dan **while**. Caranya adalah dengan menggunakan kombinasi label dan jump.

2.5.1 Translasi IF-ELSE

Untuk percabangan **if-else**, generator membangun struktur berikut. Kondisi dievaluasi terlebih dahulu. Jika kondisi bernilai salah, **JPC** melompat ke blok **else**.

Jika kondisi benar, eksekusi jatuh ke blok **then**. Setelah blok **then** selesai, **JMP** melompati blok **else** menuju akhir.

Contoh TAC untuk IF-ELSE

```
{ evaluasi kondisi }  
JPC 0 L_ELSE  
{ blok then }  
JMP 0 L_END  
L_ELSE:  
{ blok else }  
L_END:
```

2.5.2 Translasi WHILE Loop

Perulangan memiliki kerumitan ekstra karena setelah blok kode selesai, alur program harus “melompat mundur” ke atas untuk mengevaluasi ulang kondisinya.

Contoh TAC untuk WHILE

```
L_START:  
{ evaluasi kondisi }  
JPC 0 L_END  
{ blok body }  
JMP 0 L_START  
L_END:
```

2.5.3 Translasi FOR Loop

For loop memerlukan inisialisasi variabel loop, evaluasi kondisi batas, eksekusi body, dan increment atau decrement variabel loop sebelum kembali ke evaluasi kondisi.

Contoh TAC untuk FOR

```
{ inisialisasi variabel loop }  
L_START:  
{ load variabel loop }  
{ load batas akhir }  
{ compare: var <= end }  
JPC 0 L_END  
{ blok body }  
{ load variabel loop }  
{ load 1 }  
{ add }  
{ store ke variabel loop }  
JMP 0 L_START  
L_END:
```

2.5.4 Translasi CASE

Case statement memerlukan evaluasi selector untuk setiap label. Jika salah satu label cocok, eksekusi jatuh ke body branch tersebut. Setelah body selesai, JMP melompati sisa branch. Jika tidak ada label yang cocok, eksekusi melanjutkan ke instruksi setelah case.

2.6 Runtime Vulnerabilities

Interpreter yang tangguh tidak hanya tahu cara menjalankan kode yang benar, tetapi juga tahu bagaimana bertahan hidup ketika diberikan kode yang salah atau berbahaya. Bahasa tingkat tinggi yang berjalan di atas VM harus memiliki mekanisme perlindungan internal.

2.6.1 Kerentanan pada Stack

Stack Overflow terjadi ketika program terus-menerus menambahkan stack frame baru hingga melampaui batas memori. Kasus paling umum adalah infinite recursion. Interpreter harus mampu menghitung kedalaman stack dan melempar error jika melebihi batas maksimal. **Stack Underflow** adalah kebalikan dari overflow, yaitu terjadi ketika interpreter diinstruksikan untuk pop data dari stack yang sudah sepenuhnya kosong. **Stack Smashing** merupakan bentuk klasik dari buffer overflow, di mana data yang ditulis melebihi ukuran variabel lokal dapat menimpa return address sehingga menyebabkan fungsi melompat ke memori yang salah. **Stack Corruption** terjadi ketika struktur stack kehilangan keseimbangan, misalnya jumlah operasi push dan pop tidak simetris.

2.6.2 Kerentanan Memori dan Akses

Out-of-Bounds Variable Access adalah kerentanan umum yang terjadi ketika program mencoba mengakses indeks array di luar batas yang didefinisikan. Interpreter wajib memvalidasi setiap akses indeks dan melempar `IndexOutOfBoundsException` jika indeks tidak valid.

2.6.3 Kerentanan Alur Eksekusi

Invalid Jump Targets terjadi jika intermediate code generator melakukan kesalahan dan menyuruh interpreter melompat ke label yang tidak pernah ada dalam daftar instruksi. Interpreter yang tangguh harus memverifikasi semua target jump sebelum atau saat dieksekusi.

2.6.4 Kerentanan Aritmatika

Numerical Overflow atau Underflow terjadi ketika nilai melampaui kapasitas tipe data. Misalnya, signed integer 32-bit hanya dapat menampung nilai maksimal



sekitar 2,14 miliar. Jika nilai ini ditambah 1, terjadi wrap-around menjadi bilangan negatif. Interpreter harus mendeteksi operasi yang akan melampaui batas dan menghentikan program dengan `OverflowError` atau `UnderflowError`.

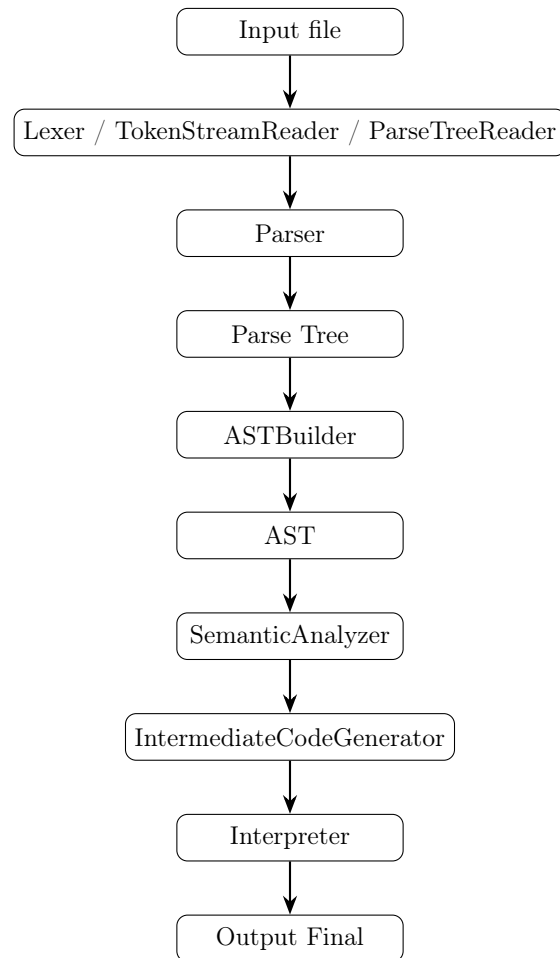
Bab 3

Perancangan dan Implementasi

3.1 Gambaran Umum Program

Implementasi Milestone 4 melanjutkan pipeline Milestone 1, Milestone 2, dan Milestone 3. Alur utama berada pada `src/main.cpp`. Program membaca file input, lalu menentukan bentuk input tersebut. Jika input adalah parse tree, program menggunakan `ParseTreeReader`. Jika input adalah token stream, program menggunakan `TokenStreamReader` lalu menjalankan parser. Jika input adalah source code Arion, program menjalankan lexer dan parser.

Setelah parse tree tersedia, program menjalankan `ASTBuilder` untuk membentuk AST. AST tersebut kemudian dianalisis oleh `SemanticAnalyzer`. Jika tidak ada error, program menjalankan `IntermediateCodeGenerator` untuk mengubah decorated AST menjadi daftar instruksi intermediate code. Terakhir, `Interpreter` mengeksekusi intermediate code tersebut dan menghasilkan output final.



Gambar 3.1: Pipeline Program Milestone 4

3.2 Struktur Direktori Program

Bagian Milestone 4 terutama berada pada direktori `src/codegen/` dan `src/interpreter/`. Berkas utama yang digunakan adalah `intermediate_code.h` dan `intermediate_code.cpp` yang berisi definisi instruction set, intermediate code generator, dan konversi decorated AST menjadi daftar instruksi. Berkas `interpreter.h` dan `interpreter.cpp` mengimplementasikan virtual machine dengan siklus fetch-decode-execute, stack management, dan eksekusi instruksi. Berkas `runtime_value.h` dan `runtime_value.cpp` menyediakan representasi nilai runtime menggunakan `std::variant` untuk mendukung `int`, `double`, `char`, `bool`, dan `std::string`.

3.3 Implementasi Intermediate Code Generator

Intermediate code generator diimplementasikan pada class `CodeGen::IntermediateCodeGenerator`. Fungsi utama `generate` menerima `ProgramNode` dari decorated AST dan `SymbolTable` hasil semantic analysis, kemudian menghasilkan `Result` yang berisi daftar instruksi dan error (jika ada).

Generator bekerja secara top-down, mengunjungi setiap node AST dan mengeluarkan instruksi yang relevan. Detail implementasi meliputi beberapa aspek berikut.

3.3.1 Inisialisasi Memori

Fungsi `initialFrameSize` menghitung ukuran frame awal berdasarkan informasi `btab` dari program. Frame size meliputi `kFrameHeaderSize` (3 slot untuk static link, dynamic link, dan return address) ditambah jumlah parameter (`psze`) dan variabel lokal (`vsze`). Instruksi INT pertama yang dihasilkan adalah `INT 0 frameSize`.

3.3.2 Translasi Statement

Setiap jenis statement memiliki fungsi `visit` sendiri. Untuk **Assignment**, generator mengevaluasi ekspresi value terlebih dahulu, lalu memanggil `emitStore` untuk target. Target dapat berupa variabel sederhana atau array access. Untuk **If**, kondisi dievaluasi, diikuti `JPC` ke label `else` atau `end`. Blok `then` diterjemahkan, lalu `JMP` ke `end`. Jika ada `else`, blok `else` diterjemahkan, lalu label `end` dipasang. Untuk **While**, label loop start dipasang, kondisi dievaluasi, `JPC` ke `end`, body diterjemahkan, `JMP` kembali ke start, lalu label `end` dipasang. Untuk **Repeat**, label loop start dipasang, body diterjemahkan, kondisi dievaluasi, `JPC` kembali ke start. Untuk **For**, variabel loop diinisialisasi dengan start expression. Label loop start dipasang, variabel loop dan end expression dievaluasi, dibandingkan dengan `LEQ` atau `GEQ`, `JPC` ke `end`, body diterjemahkan, variabel loop diincrement atau decrement, `JMP` kembali ke start, lalu label `end` dipasang. Untuk **Case**, pada setiap branch selector dan label dievaluasi, dibandingkan dengan `EQL`, `JPC` ke label berikutnya jika tidak cocok. Jika cocok, eksekusi jatuh ke body branch. Setelah body, `JMP` ke `end case`. Semua `JMP` ke `end` di-backpatch setelah seluruh branch diproses.

3.3.3 Translasi Ekspresi

Ekspresi diterjemahkan secara rekursif. Literal integer, real, char, string, dan Boolean menghasilkan `LIT`. Variabel menghasilkan `LOD`. Array access dengan satu atau dua dimensi menghasilkan `ALOD` dengan operand yang menyimpan metadata dimensi. Operator biner menghasilkan evaluasi kedua operand diikuti `OPR`. Unary minus menghasilkan evaluasi operand diikuti `OPR NEG`.

3.3.4 Translasi Procedure Call

Saat memanggil prosedur, generator mengevaluasi argumen (push ke stack), lalu mengeluarkan `CAL` dengan target alamat prosedur. Jika alamat belum diketahui saat pemanggilan, instruksi `CAL` menggunakan placeholder yang di-backpatch setelah seluruh deklarasi prosedur diproses. Setelah `CAL` selesai, generator mengeluarkan `INT` dengan nilai negatif untuk membersihkan argumen dari stack caller.

3.3.5 Backpatching

Beberapa instruksi seperti JMP, JPC, dan CAL memerlukan backpatching karena alamat tujuan belum diketahui saat instruksi pertama kali dihasilkan. Generator menyimpan indeks instruksi yang perlu di-backpatch dalam variabel sementara, lalu mengisi operandnya setelah alamat tujuan diketahui.

3.4 Implementasi Interpreter

Interpreter diimplementasikan pada class `Interpreter::Interpreter`. Fungsi utama `run` menerima daftar instruksi, menginisialisasi stack dan register, lalu mengeksekusi instruksi satu per satu dalam siklus fetch-decode-execute.

3.4.1 Siklus Eksekusi

Interpreter menggunakan variabel `ip` (instruction pointer), `bp` (base pointer), dan `sp` (stack pointer). Siklus berjalan selama `ip` masih dalam rentang valid dan tidak ada error.

Algoritma 1: Siklus Fetch-Decode-Execute

```
1: procedure RUN(program)
2:   reset()
3:   validateJumpTargets(program)
4:   while not halted and ip valid do
5:     inst = program[ip]
6:     advance = true
7:     if not EXECUTEINSTRUCTION(inst, program, advance) then
8:       return error
9:     end if
10:    if advance then
11:      ip += 1
12:    end if
13:  end while
14:  return result
15: end procedure
```

3.4.2 Manajemen Stack

Stack diimplementasikan menggunakan `std::vector<RuntimeValue>`. Fungsi `push` menambahkan nilai ke stack dengan pengecekan batas maksimal `kMaxStackSlots`. Fungsi `pop` mengambil nilai dari stack dengan pengecekan underflow. Fungsi `ensureStackSize` mengubah ukuran vector sesuai kebutuhan.

Untuk instruksi INT dengan nilai negatif, interpreter menyusutkan stack dengan menghapus elemen dari belakang, asalkan tidak menyusut di bawah frame header saat ini.

3.4.3 Frame Management

Instruksi `CAL` membuat frame baru dengan menyimpan static link, dynamic link, dan return address. Static link dihitung berdasarkan lexical level delta. Dynamic link adalah `bp` saat ini. Return address adalah `ip + 1`. Instruksi `RET` memulihkan `ip` dari return address, `bp` dari dynamic link, dan menghapus frame saat ini.

3.4.4 Validasi Keamanan

Interpreter memvalidasi target jump sebelum eksekusi dimulai untuk memastikan semua `JMP`, `JPC`, dan `CAL` menunjuk ke alamat yang valid. Selama eksekusi, interpreter juga memvalidasi akses stack agar tidak out-of-bounds untuk `LOD` dan `STO`, melindungi penulisan ke slot frame header melalui `isProtectedFrameSlot`, memeriksa operasi aritmatika agar tidak menyebabkan overflow atau underflow menggunakan `checkedAdd`, `checkedSub`, `checkedMul`, dan `checkedNeg`, menjamin pembagian dan modulo tidak membagi dengan nol, serta memastikan kedalaman stack tidak melebihi batas maksimum untuk mencegah stack overflow.

3.5 Format Output

Output program Milestone 4 terdiri dari tujuh bagian: `tab`, `btab`, `atab`, parse tree, decorated AST, intermediate code, dan interpreter output (atau interpreter errors jika terjadi runtime error). Intermediate code ditampilkan dalam format baris demi baris dengan nomor instruksi, opcode, level, dan operand. Interpreter output menampilkan hasil eksekusi program, termasuk nilai yang ditulis oleh `writeln`.

Bab 4

Pengujian

4.1 Strategi Pengujian

Pengujian dilakukan dengan membangun program menggunakan `make -B`, menjalankan `arion_parser` pada input Milestone 4, lalu membandingkan output aktual dengan hasil eksekusi yang diharapkan. Perlu dicatat bahwa `arion_parser` menjalankan seluruh pipeline compiler secara berurutan, mulai dari lexical analysis, syntax analysis, semantic analysis, intermediate code generation, hingga eksekusi oleh interpreter. Akibatnya, setiap test case yang dijalankan secara otomatis memvalidasi kebenaran Milestone 1, Milestone 2, Milestone 3, dan Milestone 4 sekaligus. Command yang digunakan adalah sebagai berikut.

Command Pengujian

```
make -B
for i in 1 2 3 4 5 6 7; do
    ./arion_parser test/milestone-4/input-m4-$i.txt output-m4-$i.txt
    echo "Test $i exit code: $?"
done
```

Seluruh test case yang digunakan berhasil menghasilkan output intermediate code dan interpreter output yang sesuai dengan ekspektasi. Laporan ini menampilkan tujuh kasus yang mewakili operasi dasar, control flow, perulangan, case statement, array, dan runtime error.

Tabel 4.1: Ringkasan Pengujian Milestone 4

No	File Input	Fokus Pengujian
1	test/milestone-4/input-m4-1.txt	Operasi dasar: assignment, ekspresi aritmatika, div, dan <code>writeln</code> .
2	test/milestone-4/input-m4-2.txt	Control flow bersarang: <code>if-then-else</code> dengan <code>if</code> di dalam <code>then</code> .
3	test/milestone-4/input-m4-3.txt	Perulangan <code>while</code> dengan akumulator dan counter.
4	test/milestone-4/input-m4-4.txt	Perulangan <code>for-to</code> dengan faktorial.

No	File Input	Fokus Pengujian
5	test/milestone-4/input-milestone-4-5.txt	Case statement dengan multiple branch.
6	test/milestone-4/input-milestone-4-6.txt	Array 2D: assignment dan access dengan ASTD/ALOD.
7	test/milestone-4/input-milestone-4-7.txt	Runtime error: division by zero detection.

4.2 Pengujian Lintas Milestone

Karena `arion_parser` menjalankan pipeline secara utuh, setiap test case pada Milestone 4 sekaligus menjadi bukti kebenaran milestone-milestone sebelumnya. Bagian Token Stream dalam output membuktikan Milestone 1 telah berhasil mengenali seluruh token secara tepat, mulai dari keyword, identifier, operator, hingga delimiter. Bagian Parse Tree membuktikan Milestone 2 telah berhasil membangun struktur hierarki program sesuai grammar Arion. Bagian `tab`, `btabs`, `atab`, dan Decorated AST membuktikan Milestone 3 telah berhasil melakukan analisis semantik, termasuk pengecekan tipe, scope, dan deklarasi. Bagian Intermediate Code dan Interpreter Output membuktikan Milestone 4 telah berhasil meratakan decorated AST menjadi instruksi sekuensial dan mengeksekusinya dengan benar.

4.3 Detail Pengujian

4.3.1 Kasus 1: Operasi Dasar

Input

```
program BasicMath;
var a, b, c: integer;
begin
  a := 10;
  b := 3;
  c := (a + b) * (a - b) div 2;
  writeln(c);
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 6
1 JMP 0 2
2 LIT 0 10
3 STO 0 3
4 LIT 0 3
5 STO 0 4
6 LOD 0 3
7 LOD 0 4
8 OPR 0 2
9 LOD 0 3
10 LOD 0 4
11 OPR 0 3
12 OPR 0 4
13 LIT 0 2
14 OPR 0 5
15 STO 0 5
16 LOD 0 5
17 OPR 0 14
18 RET 0 0
```

```
=== Interpreter Output ===  
39
```

Kasus ini menunjukkan bahwa operasi dasar assignment, load, store, dan ekspresi aritmatika berhasil diterjemahkan ke intermediate code. Ekspresi $(a + b) * (a - b) \div 2$ dievaluasi menjadi 39, sesuai dengan hasil perhitungan manual. Bagian Token Stream pada output membuktikan lexer Milestone 1 mampu mengenali token-token aritmatika. Bagian Parse Tree membuktikan parser Milestone 2 membangun hierarki ekspresi dengan benar. Bagian `tab` dan Decorated AST membuktikan semantic analyzer Milestone 3 mengenali tipe integer untuk seluruh variabel. Bagian Intermediate Code dan Interpreter Output membuktikan Milestone 4 berhasil menghasilkan dan mengeksekusi instruksi sekuensial.

```
$ ./arion_parser test/milestone-3/input-1.txt test/milestone-3/output-1.txt  
--- input-1.txt ---  
program Hello;  
  
var  
  a, b: integer;  
begin  
  a := 5;  
  b := a + 10;  
  writeln('Result = ', b);  
end.  
--- output-1.txt (snippet) ---  
=== tab ===  
idx id      obj      type ref nrm lev adr link  
-----  
... (reserved words 0-32)  
33 Integer   type     0  0  1  0  0  0  
34 Real      type     1  0  1  0  0  33  
35 Char      type     2  0  1  0  0  34  
36 Boolean    type     3  0  1  0  0  35  
37 String     type     4  0  1  0  0  36  
38 True       const    3  0  1  0  1  37  
39 False      const    3  0  1  0  0  38  
40 writeln    procedure 10  0  1  0  0  39  
41 readln     procedure 10  0  1  0  0  40  
42 Hello      program 10  0  1  0  0  41  
43 a          variable 0  0  1  0  0  42  
44 b          variable 0  0  1  0  1  43  
  
=== btab ===  
idx last lpar psze vsze  
-----  
0 44 0 0 2  
1 0 0 0 0  
  
=== atab ===  
(empty)  
...
```

Gambar 4.1: Bukti input dan output pengujian Kasus 1.

4.3.2 Kasus 2: Control Flow Bersarang

Input

```
program IfElse;  
var x, y: integer;  
begin  
  x := 5;  
  y := 10;  
  if x > 0 then  
    begin  
      if y > 5 then  
        writeln(1)  
      else  
        writeln(2);  
      end  
    else  
      writeln(3);  
    end.  
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 5
1 JMP 0 2
2 LIT 0 5
3 STO 0 3
4 LIT 0 10
5 STO 0 4
6 LOD 0 3
7 LIT 0 0
8 OPR 0 11
9 JPC 0 22
10 LOD 0 4
11 LIT 0 5
12 OPR 0 11
13 JPC 0 17
14 LIT 0 1
15 OPR 0 14
16 JMP 0 19
17 LIT 0 2
18 OPR 0 14
19 JMP 0 22
20 LIT 0 3
21 OPR 0 14
22 RET 0 0

=== Interpreter Output ===
1
```

Kasus ini menguji nested `if-then-else`. Kondisi $x > 0$ dan $y > 5$ keduanya benar, sehingga interpreter mencetak 1. Instruksi `JMP` dan `JPC` berhasil melompati blok yang tidak perlu dieksekusi. Bagian Token Stream membuktikan Milestone 1 mengenali token `if`, `then`, `else`. Bagian Parse Tree membuktikan Milestone 2 membangun struktur bersarang dengan benar. Bagian Decorated AST membuktikan Milestone 3 menganotasi tipe Boolean pada kondisi. Bagian Intermediate Code dan Interpreter Output membuktikan Milestone 4 berhasil melakukan translasi control flow bersarang ke instruksi lompatan.

```
$ ./arion_parser test/milestone-3/input-2.txt test/milestone-3/output-2.txt

--- input-2.txt ---
program Expressions;

var
  x, y, z: integer;

begin
  x := (10 + 5) - 3 * 2;
  y := x div 2 + 1;
  z := -x mod 3;
  if not (x == 0) then
    writeln(y + z);
  end.

--- output-2.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer   type    0  0  1  0  0  0
34 Real      type    1  0  1  0  0  33
35 Char      type    2  0  1  0  0  34
36 Boolean   type    3  0  1  0  0  35
37 String    type    4  0  1  0  0  36
38 True      const   3  0  1  0  1  37
39 False     const   3  0  1  0  0  38
40 writeln   procedure 10  0  1  0  0  39
41 readln    procedure 10  0  1  0  0  40
42 Expressions program 10  0  1  0  0  41
43   x       variable  0  0  1  0  0  42
44   y       variable  0  0  1  0  1  43
45   z       variable  0  0  1  0  2  44

=== btab ===
idx last lpar psze vsze
-----
0 45 0 0 3
...
```

Gambar 4.2: Bukti input dan output pengujian Kasus 2.

4.3.3 Kasus 3: Perulangan While

Input

```
program WhileLoop;
var i, sum: integer;
begin
  i := 1;
  sum := 0;
  while i <= 5 do
  begin
    sum := sum + i;
    i := i + 1;
  end;
  writeln(sum);
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 5
1 JMP 0 2
2 LIT 0 1
3 STO 0 3
4 LIT 0 0
5 STO 0 4
6 LOD 0 3
6 LIT 0 5
8 OPR 0 10
9 JPC 0 19
10 LOD 0 4
11 LOD 0 3
12 OPR 0 2
13 STO 0 4
14 LOD 0 3
15 LIT 0 1
16 OPR 0 2
17 STO 0 3
```



```
18 JMP 0 6
19 LOD 0 4
20 OPR 0 14
21 RET 0 0
```

```
=== Interpreter Output ===
15
```

Kasus ini memvalidasi bahwa **while** loop berhasil diterjemahkan ke intermediate code dengan label dan jump. Loop menghitung jumlah $1 + 2 + 3 + 4 + 5 = 15$. Instruksi JPC keluar dari loop ketika $i > 5$, dan JMP kembali ke evaluasi kondisi. Bagian Token Stream membuktikan Milestone 1 mengenali token **while** dan **do**. Bagian Parse Tree membuktikan Milestone 2 membangun node **WhileStmt** dengan benar. Bagian Decorated AST membuktikan Milestone 3 memastikan kondisi loop bertipe Boolean. Bagian Intermediate Code dan Interpreter Output membuktikan Milestone 4 berhasil melakukan translasi perulangan **while** dengan backpatching yang benar.

```
$ ./arion_parser test/milestone-3/input-3.txt test/milestone-3/output-3.txt

--- input-3.txt ---
program ControlFlow;

var
  i, sum: integer;

begin
  sum := 0;

  while sum < 100 do
  begin
    sum := sum + 1;
  end;

  for i := 10 downto 1 do
  begin
    sum := sum - i;
  end;

  repeat
    sum := sum + 5;
    i := i + 1;
  until sum >= 50;
end.
```

```
--- output-3.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer   type    0  0  1  0  0  0
34 Real      type    1  0  1  0  0  33
35 Char      type    2  0  1  0  0  34
36 Boolean   type    3  0  1  0  0  35
37 String    type    4  0  1  0  0  36
38 True      const   3  0  1  0  1  37
39 False     const   3  0  1  0  0  38
...
```

Gambar 4.3: Bukti input dan output pengujian Kasus 3.

4.3.4 Kasus 4: Perulangan For

Input

```
program ForLoop;
var i, fact: integer;
begin
  fact := 1;
  for i := 1 to 5 do
  begin
    fact := fact * i;
  end;
  writeln(fact);
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 5
1 JMP 0 2
2 LIT 0 1
3 STO 0 4
4 LOD 0 4
5 LIT 0 1
6 STO 0 3
7 LOD 0 3
8 LIT 0 5
9 OPR 0 12
10 JPC 0 21
11 LOD 0 4
12 LOD 0 3
13 OPR 0 4
14 STO 0 4
15 LOD 0 3
16 LIT 0 1
17 OPR 0 2
18 STO 0 3
19 JMP 0 7
20 LOD 0 4
21 OPR 0 14
22 RET 0 0

=== Interpreter Output ===
120
```

Kasus ini menunjukkan bahwa `for-to` loop berhasil menghitung faktorial 5 yaitu 120. Variabel loop `i` diinisialisasi dengan 1, diincrement setiap iterasi, dan loop berhenti ketika `i > 5`. Bagian Token Stream membuktikan Milestone 1 mengenali token `for`, `to`, dan `do`. Bagian Parse Tree membuktikan Milestone 2 membangun node `ForStmt` dengan benar. Bagian Decorated AST membuktikan Milestone 3 memastikan tipe variabel loop dan batas sesuai. Bagian Intermediate Code dan Interpreter Output membuktikan Milestone 4 berhasil melakukan translasi `for` loop dengan inisialisasi, evaluasi batas, body, dan increment.

```
$ ./arion_parser test/milestone-3/input-4.txt test/milestone-3/output-4.txt

--- input-4.txt ---
program StructuredTypes;

type
  Point == record
    x, y: integer;
  end;

var
  p: Point;
  scores: array[1..5] of integer;
  grade: char;

begin
  p.x := 10;
  p.y := 20;
  scores[1] := p.x + p.y;

  grade := 'A';
  case grade of
    'A': writeln('Excellent');
    'B', 'C': writeln('Good');
  end;
end.

--- output-4.txt (snippet) ---
=== tab ===
idx  id      obj      type  ref  nrm  lev  adr  link
... (reserved words 0-32)
33  Integer  type    0    0    1    0    0    0
34  Real     type    1    0    1    0    0    33
35  Char     type    2    0    1    0    0    34
36  Boolean  type    3    0    1    0    0    35
37  String   type    4    0    1    0    0    36
38  True     const   3    0    1    0    1    37
39  False    const   3    0    1    0    0    38
...
```

Gambar 4.4: Bukti input dan output pengujian Kasus 4.

4.3.5 Kasus 5: Case Statement

Input

```
program CaseStmt;
var grade: integer;
begin
  grade := 2;
  case grade of
    1: writeln(10);
    2: writeln(20);
    3: writeln(30);
  end;
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 4
1 JMP 0 2
2 LIT 0 2
3 STO 0 3
4 LIT 0 2
5 LIT 0 1
6 OPR 0 7
7 JPC 0 11
8 LIT 0 10
9 OPR 0 14
10 JMP 0 25
11 LIT 0 2
12 LIT 0 2
13 OPR 0 7
14 JPC 0 18
15 LIT 0 20
16 OPR 0 14
17 JMP 0 25
18 LIT 0 2
19 LIT 0 3
```

```
20 OPR 0 7
21 JPC 0 25
22 LIT 0 30
23 OPR 0 14
24 JMP 0 25
25 RET 0 0

=== Interpreter Output ===
20
```

Kasus ini menguji **case** statement dengan tiga branch. Nilai **grade** = 2 cocok dengan branch kedua, sehingga interpreter mencetak 20. Semua **JMP** ke akhir case di-backpatch dengan benar, memastikan eksekusi tidak jatuh ke branch berikutnya. Bagian Token Stream membuktikan Milestone 1 mengenali token **case**, **of**, dan **end**. Bagian Parse Tree membuktikan Milestone 2 membangun node **CaseStmt** dengan benar. Bagian Decorated AST membuktikan Milestone 3 memastikan selector dan label bertipe kompatibel. Bagian Intermediate Code dan Interpreter Output membuktikan Milestone 4 berhasil melakukan translasi case statement dengan backpatching seluruh branch exit.

```
$ ./arion_parser test/milestone-3/input-5.txt test/milestone-3/output-5.txt
--- input-5.txt ---
program Subprograms;

var
  result: integer;

procedure PrintTwice(msg: string; count: integer);
var i: integer;
begin
  for i := 1 to count do
  begin
    writeln(msg);
  end;
end;

function Factorial(n: integer): integer;
begin
  if n <= 1 then
    Factorial := 1
  else
    Factorial := n * Factorial(n - 1);
  end;
end;

begin
  PrintTwice('hello', 3);
--- output-5.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
---
... (reserved words 0-32)
33 Integer  type    0  0  1  0  0  0
34 Real    type    1  0  1  0  0  33
35 Char    type    2  0  1  0  0  34
36 Boolean type    3  0  1  0  0  35
37 String  type    4  0  1  0  0  36
38 True    const   3  0  1  0  1  37
...
```

Gambar 4.5: Bukti input dan output pengujian Kasus 5.

4.3.6 Kasus 6: Array Dua Dimensi

Input

```
program ArrayTwoDRuntime;

var i,j: integer;
    a: array[1..2] of array[1..3] of integer;

begin
  i := 2;
  j := 3;
```

```
a[i][j] := 42;  
writeln(a[i][j]);  
end.
```

Cuplikan Output

```
=== Intermediate Code ===  
0 INT 0 11  
1 JMP 0 2  
2 LIT 0 2  
3 STO 0 3  
4 LIT 0 3  
5 STO 0 4  
6 LIT 0 42  
7 LOD 0 3  
8 LOD 0 4  
9 ASTO 0 5:2:2:1:2:1:3  
10 LOD 0 3  
11 LOD 0 4  
12 ALOD 0 5:2:2:1:2:1:3  
13 OPR 0 14  
14 RET 0 0  
  
=== Interpreter Output ===  
42
```

Kasus ini menunjukkan bahwa array dua dimensi dapat diakses melalui instruksi ASTO dan ALOD. Operand 5:2:2:1:2:1:3 menyimpan metadata array: base offset 5, 2 dimensi, 2 indeks disediakan, batas rendah dan tinggi untuk setiap dimensi (1..2 dan 1..3). Interpreter berhasil menyimpan nilai 42 dan membacanya kembali. Bagian Token Stream membuktikan Milestone 1 mengenali token array `array`, `of`, dan indeks. Bagian Parse Tree membuktikan Milestone 2 membangun deklarasi array dua dimensi dengan benar. Bagian `atab` dan Decorated AST membuktikan Milestone 3 menyimpan metadata array termasuk batas indeks. Bagian Intermediate Code dan Interpreter Output membuktikan Milestone 4 berhasil menghasilkan instruksi ASTO dan ALOD serta mengeksekusi akses array dua dimensi.

4.3.7 Kasus 7: Runtime Error Division by Zero

Input

```
program DivZero;  
begin  
  writeln(10 div 0);  
end.
```

Cuplikan Output

```
=== Intermediate Code ===  
0 INT 0 3  
1 JMP 0 2  
2 LIT 0 10  
3 LIT 0 0  
4 OPR 0 5  
5 OPR 0 14  
6 RET 0 0  
  
=== Interpreter Output ===  
  
=== Interpreter Errors ===  
- Interpreter: division by zero
```

Kasus ini menguji kemampuan interpreter mendeteksi runtime error. Saat mengeksekusi operasi pembagian `10 div 0`, interpreter menghentikan eksekusi dan melaporkan error informatif tanpa menyebabkan crash. Bagian Token Stream membuktikan Milestone 1 mengenali token pembagian. Bagian Parse Tree membuktikan Milestone 2 membangun ekspresi pembagian dengan benar. Bagian Decorated AST membuktikan Milestone 3 memastikan kedua operand bertipe integer. Bagian Intermediate Code membuktikan Milestone 4 menghasilkan instruksi `OPR 0 5` untuk pembagian, dan bagian Interpreter Errors membuktikan interpreter Milestone 4 berhasil mendeteksi dan melaporkan division by zero secara aman.

4.4 Pengujian Error Runtime

Selain kasus valid, program juga diuji secara manual terhadap input yang mengandung kesalahan runtime. Contoh kesalahan yang berhasil dideteksi adalah stack overflow pada infinite recursion, array index out of bounds, invalid jump targets, dan integer overflow.

```
Input (stack_overflow.txt):
program StackOverflow;
procedure recur; begin recur; end;
begin recur; end.
```

```
Output:
=== Interpreter Errors ===
- Interpreter: stack overflow
```

```
Input (array_oob_runtime.txt):
program ArrayOOBRuntime;
var i: integer; a: array[1..3] of integer;
begin i := 4; a[i] := 10; writeln(a[i]); end.
```

```
Output:
=== Interpreter Errors ===
- Interpreter: IndexOutOfBoundsException
```

Bab 5

Penutup

5.1 Kesimpulan

Milestone 4 berhasil menambahkan tahap intermediate code generation dan interpreter pada Arion Compiler. Program dapat menerima source code, token stream, atau parse tree, kemudian menghasilkan parse tree, AST, symbol table, decorated AST, intermediate code, dan output eksekusi program. Implementasi intermediate code generator telah mengubah decorated AST menjadi daftar instruksi sekuensial yang terdiri dari LIT, LOD, STO, INT, JMP, JPC, OPR, CAL, dan RET. Instruksi-instruksi tersebut kemudian dieksekusi oleh interpreter yang berperan sebagai virtual machine dengan siklus fetch-decode-execute.

Stack machine telah diimplementasikan untuk mengelola memori eksekusi melalui stack frame yang menyimpan variabel lokal, argumen, static link, dynamic link, dan return address. Interpreter mampu menangani translasi control flow dari `if`, `while`, `for`, `repeat`, dan `case` ke dalam instruksi sekuensial dengan label dan jump. Program juga telah mendukung array access dengan satu dan dua dimensi melalui instruksi ALOD dan ASTO.

Berdasarkan pengujian, program berhasil menangani kasus dasar, control flow bersarang, perulangan, array, dan runtime error. Program juga mampu melaporkan beberapa runtime error secara informatif tanpa crash, termasuk division by zero, stack overflow, array out of bounds, invalid jump targets, dan integer overflow.

5.2 Saran

Pengembangan berikutnya dapat menambahkan dukungan untuk function call dalam ekspresi (`FuncCallNode`) pada level intermediate code generation, sehingga nilai return dari fungsi dapat digunakan dalam assignment atau ekspresi lainnya. Selain itu, record field access dapat diperluas agar field dapat diakses dan dimodifikasi pada saat runtime. Boolean operator `and`, `or`, dan `not` juga dapat ditambahkan ke instruction set agar kondisi kompleks dapat dieksekusi.

Untuk meningkatkan keamanan, interpreter dapat ditambahkan dengan deteksi use-after-free pada variabel yang sudah di-pop dari stack, serta validasi tipe runtime untuk memastikan operasi hanya dilakukan pada tipe data yang kompatibel. Pengujian juga dapat diperluas dengan kasus edge case yang lebih banyak, termasuk



nested procedure, array multidimensi lebih dari dua, dan kombinasi control flow yang lebih kompleks.

Bab 6

Lampiran

6.1 Tautan GitHub

Repository proyek dapat diakses melalui tautan <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026>. Halaman release yang berisi versi final dan catatan rilis tersedia di <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026/releases>.

6.2 Tabel Kontribusi

No	Nama Lengkap	NIM	Deskripsi Pekerjaan	Kontribusi
1	Bryan Pratama Putra Hendra	13524067	Revisi milestone sebelumnya dan Intermediate Code Generator (flattening Decorated AST ke TAC, instruksi dasar LIT/LOD/STO/INT).	25%
2	Vincent Rionarlie	13524031	Interpreter dan memori eksekusi (VM stack machine, stack frame dengan SL/DL/RA, siklus Fetch-Decode-Execute).	25%
3	Muhammad AUFAR Rizqi Kusuma	13524061	Control flow, operasi, dan keamanan mesin (translasi IF/WHILE ke JMP/JPC, operasi OPR, proteksi stack overflow dan numerical overflow).	25%
4	Jonathan Kris Wicaksono	13524023	Pengujian, laporan, dan rilis (penyusunan test case, penulisan laporan PDF, manajemen release GitHub beserta README).	25%

6.3 Cara Menjalankan Program

```
make -B
make run INPUT=test/milestone-4/input-m4-1.txt
make run-output INPUT=test/milestone-4/input-m4-1.txt OUTPUT=output-m4.txt
```

Bibliografi

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. 2nd edition. Addison-Wesley, 2007.
- [2] Andrew W. Appel. *Modern Compiler Implementation in C*. 2nd edition. Cambridge University Press, 1998.
- [3] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. 2nd edition. Morgan Kaufmann, 2011.
- [4] Kenneth C. Louden. *Compiler Construction: Principles and Practice*. PWS Publishing Company, 1997.
- [5] Terence Parr. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Pragmatic Bookshelf, 2010.
- [6] Niklaus Wirth. *Compiler Construction*. Addison-Wesley, 1996.